

LAB 9

BANKER'S ALGORITHM

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int n, m;
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    printf("Enter the number of Resource types: ");
    scanf("%d", &m);
    int alloc[n][m], max[n][m], avail[m], need[n][m];
    int f[n], ans[n], ind = 0;
    printf("\nEnter the Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process P%d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    }
    printf("\nEnter the Max Matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process P%d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    }
    printf("\nEnter the Available Resources: ");
    for (int i = 0; i < m; i++)
        scanf("%d", &avail[i]);
    for (int k = 0; k < n; k++) f[k] = 0;
    for (int i = 0; i < n; i++) {
```

```

    for (int j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];
}
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        if (f[i] == 0) {
            int flag = 0;
            for (int j = 0; j < m; j++) {
                if (need[i][j] > avail[j]) {
                    flag = 1;
                    break;
                }
            }
            if (flag == 0) {
                ans[ind++] = i;
                for (int y = 0; y < m; y++)
                    avail[y] += alloc[i][y];
                f[i] = 1;
            }
        }
    }
}

printf("\n--- Result ---\n");

bool safe = true;
for (int i = 0; i < n; i++) {
    if (f[i] == 0) {
        safe = false;
        printf("The system is in an UNSAFE state (Deadlock possible).\n");
        break;
    }
}

```

```

    }

    if (safe) {

        printf("The system is in a SAFE state.\nSafe Sequence: ");

        for (int i = 0; i < n - 1; i++)

            printf("P%d -> ", ans[i]);

        printf("P%d\n", ans[n - 1]);

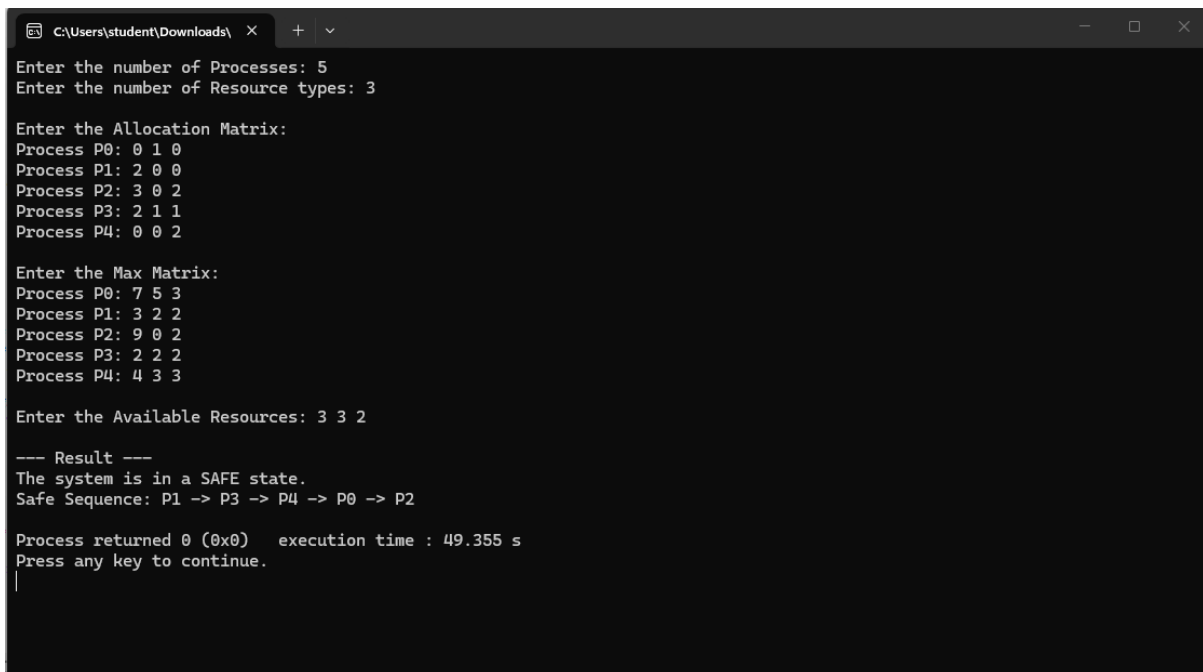
    }

    return 0;

}

```

OUTPUT



```

C:\Users\student\Downloads\ >
Enter the number of Processes: 5
Enter the number of Resource types: 3

Enter the Allocation Matrix:
Process P0: 0 1 0
Process P1: 2 0 0
Process P2: 3 0 2
Process P3: 2 1 1
Process P4: 0 0 2

Enter the Max Matrix:
Process P0: 7 5 3
Process P1: 3 2 2
Process P2: 9 0 2
Process P3: 2 2 2
Process P4: 4 3 3

Enter the Available Resources: 3 3 2

--- Result ---
The system is in a SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 49.355 s
Press any key to continue.
|

```